

Milestone P1 – Initial Experiment and Evaluation Setup

Project Title:

RECAP – Rapid Event-level Classification of Affected Properties

Authors:

Yatin Bhupatiraju

Aryan Anand

Sriradhaa Satishkumar

University of South Carolina

Emails: yatinrcb@gmail.com , sriradhaas@gmail.com , A.aryansurya007@gmail.com

System and Dataset Setup

The RECAP system is developed in Python 3.11 using PyTorch 2.4.1 on a Windows 11 workstation equipped with an AMD Ryzen 7 5700X (8 cores) CPU and an AMD RX 7800 XT GPU accelerated through DirectML.

The primary dataset used is a **processed subset of the xView2 challenge dataset**, containing **building-level “before” and “after” satellite image pairs** annotated with 4 categorical damage levels:

- **No damage**
- **Minor damage**
- **Major damage**
- **Destroyed**

Each sample consists of two 224×224 RGB image crops (pre-disaster and post-disaster) combined into a **6-channel tensor**, allowing the model to directly compare pre/post visual differences.

To improve efficiency, all building crops were **precomputed and saved as .npz files** (`data/chips/{building_id}.npz`), each containing a stacked pre+post tensor. This eliminates repetitive cropping during training and speeds up both training and inference by 3–5×.

Future work will expand the dataset to the full **131 GB xBD dataset**, which includes geographic coordinates (lat/lon) for real-world mapping in the Streamlit app.

Baseline Implementation

The baseline model is a **modified ResNet-18** from torchvision, adapted for 6 input channels (3 pre + 3 post) and a 4-class output.

Architecture Summary:

Component	Implementation
Base Model	ResNet-18 (no pre-trained weights)
Input Modification	Conv1 layer modified to accept 6 channels
Output Layer	Fully connected layer → 4 softmax logits
Optimizer	AdamW (learning rate 1e-4, weight decay 1e-2)
Scheduler	CosineAnnealingLR
Loss	Focal Loss ($\gamma = 2.0$) with class weights
Data Augmentation	Random flips, rotation ($\pm 15^\circ$), color jitter, random erasing
Training Hardware	AMD RX 7800 XT via DirectML backend
Dataset Split	80% training / 20% validation

Key Features

- **6-channel direct comparison:** allows the model to focus on post-disaster changes.
- **WeightedRandomSampler:** balances the dataset to counteract heavy class imbalance (most samples are “no damage”).
- **Focal Loss:** focuses the gradient on difficult minority classes, improving recall for “major” and “destroyed” categories.
- **Checkpointing:** model, optimizer, and scheduler states are saved each epoch (recap/models/checkpoint_epoch*.pth).
- **Logging:** training loss, accuracy, validation accuracy, and macro-F1 are recorded each epoch in info/training_log.csv.

The overall training command is reproducible with:

```
python train_baseline.py
```

Preliminary Experiment

Experimental Setup

Parameter	Value
Model	ResNet-18 (6-channel)
Epochs	25
Batch Size	48
Learning Rate	1e-4
Optimizer	AdamW
Loss	Focal Loss + Class Weights
Sampler	WeightedRandomSampler
Validation Split	20%
Device	DirectML (AMD RX 7800 XT)

The model was trained for **25 epochs** on the full xView2 subset (~159 k buildings). Training was stable and converged smoothly under the cosine learning-rate schedule.

Results Summary (Epoch 25)

Metric	Value
Training Accuracy	96.47 %
Validation Accuracy	85.12 %
Macro F1 Score	0.74
Loss	0.134

Per-Class Metrics

Class	Precision	Recall	F1
No damage	0.96	0.89	0.92
Minor damage	0.48	0.64	0.55
Major damage	0.66	0.74	0.70
Destroyed	0.79	0.83	0.81

Observations

- The model performs **best on “no” and “destroyed”** classes.
- **Minor damage** remains the weakest class due to subtle pixel differences and limited examples.

- The use of **WeightedRandomSampler + Focal Loss** significantly improved recall on rare classes compared to the earlier plain CrossEntropy baseline (macro F1 $\approx$ 0.64).
- No severe overfitting observed: validation accuracy plateaued around 85%, while training approached 96%.

Visualization and Evaluation Tools

A custom **Streamlit web app** (`recap_app.py`) was built for visual evaluation and demo deployment.

Key Features:

- Interactive **map or pixel-space scatter** (toggle for datasets without lat/lon).
- **Filter by event** and **confidence threshold**.
- **Building inspector**: side-by-side pre/post images with predicted label and Grad-CAM overlay.
- **Inspection queue**: prioritizes top-confidence “destroyed” or “major” buildings.
- **Metrics tab**: displays confusion matrix, per-class precision/recall/F1, and calibration curves.
- **Download tools**: export filtered predictions or inspection queues as CSVs.

Example output files:

- `info/predictions.csv` — per-building predictions.
- `info/predictions_with_centroids.parquet` — same data with pixel centroids for app visualization.

Documentation and Reproducibility

All experiments are version-controlled and reproducible using:

```
python train_baseline.py
python -m recap.etl.predict_all
streamlit run recap_app.py
```

Reproducibility Practices:

- Fixed random seeds
- Full model checkpointing
- Training logs and CSV exports per run
- Precomputed chips for consistent inputs

Future Work

- Expand to full **xBD dataset** with true georeferenced lat/lon coordinates for world-map visualization.
- Experiment with larger backbones (ResNet-34, EfficientNet, or ViT) for higher accuracy.
- Integrate **spatial calibration and ensembling** for uncertainty estimation.
- Evaluate generalization across disaster events (train on some events, test on unseen ones).

References

1. Gupta, R., Hosfelt, R., Sajeed, S., et al. (2019). *xBD: A Dataset for Assessing Building Damage from Satellite Imagery*. CVPR Workshop on AI for Disaster Response.
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR.
3. Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). *Focal Loss for Dense Object Detection*. ICCV.
4. Streamlit Inc. (2023). *Streamlit Documentation*. [Online]. Available: <https://streamlit.io>
5. PyTorch Foundation. (2024). *PyTorch DirectML Backend*. [Online]. Available: <https://learn.microsoft.com/en-us/windows/ai/directml/>